

Project 3 - DSA 8020

Saransh Rakshak

Due: Thursday May 1, 2025

Contents

Project Description	2
Problem 1. Analyzing Yearly Average Temperatures Time-Series	2
1. The following steps are needed to extract the annual mean temperatures at the specified location from the original dataset with daily temperatures at thousands of locations:	2
a. First, use the following R script to load the R data object:	2
b. Second, choose a location of interest (with the information of longitude and latitude) and identify the nearest ERA grid cell to the chosen location. Below you can find a script to get the correct ERA grid cell for Clemson:	2
c. Third, aggregate the daily data to the annual average time series. You could use the following script to achieve this:	3
2. Describe the main features of the annual time series data you extracted.	4
3. Is it reasonable to assume that there is a linear trend? If so, estimate and remove the trend to get the detrended time series.	5
4. Make acf and pacf plots for the detrended time series to identify possible ARMA models. That is, determine the possible orders p for the <i>AR</i> component and the orders q for the <i>MA</i> component.	8
5. Fit the possible ARMA models you identified and perform model selection to determine the ‘best’ model.	10
6. Perform a one-year-ahead forecast. Calculate the prediction and provide a 95% Prediction Interval.	11
Problem 2. Analyzing Monthly Average Temperature Time Series	12
Monthly Average	12
2. Describe the main features of the monthly time series data you extracted.	12
3. Is it reasonable to assume that there is a linear trend? If so, estimate and remove the trend to get the detrended time series.	13
4. Make acf and pacf plots for the detrended time series to identify possible ARMA models. That is, determine the possible orders p for the <i>AR</i> component and the orders q for the <i>MA</i> component.	15
5. Fit the possible ARMA models you identified and perform model selection to determine the ‘best’ model.	17

Project Description

The ERA-Interim is a global atmospheric reanalysis dataset. Reanalysis refers to the process of producing spatially and temporally gridded datasets through data assimilation, primarily for climate monitoring and analysis. In the R data file `NA_MaxT_ERA.RData`, you will find daily maximum temperature values from 1979 to 2017, with a spatial resolution of approximately 80 km. The original global dataset has been subsetted to a region covering the contiguous United States and extending into Canada and Mexico.

In Problems 1 and 2, you are required to analyze an ERA-Interim average temperature time series at yearly and monthly temporal resolutions at a spatial location of your choice.

Problem 1. Analyzing Yearly Average Temperatures Time-Series

1. The following steps are needed to extract the annual mean temperatures at the specified location from the original dataset with daily temperatures at thousands of locations:

a. First, use the following R script to load the R data object:

```
load("NA_MaxT_ERA.RData")
NA_MaxT <- NA_MaxT - 273.15 # change from Kelvin to Celsius
nlon <- length(NA_lon); nlat <- length(NA_lat)
nmon <- 12; nyr <- 39 # numbers of month/year
```

b. Second, choose a location of interest (with the information of longitude and latitude) and identify the nearest ERA grid cell to the chosen location. Below you can find a script to get the correct ERA grid cell for Clemson:

```
# Clemson Example
Clemson.lon.lat <- c(-82.8374, 34.6834) # (lon,lat)
dist2Clemson <- rdist.earth(
  matrix(Clemson.lon.lat, 1, 2),
  expand.grid(NA_lon, NA_lat),
  miles = F
)
id <- which.min(dist2Clemson)
(lon_id <- id %% nlon)
```

```
## [1] 58
```

```
(lat_id <- id %/% nlon + 1)
```

```
## [1] 15
```

```
# Check
(rdist.earth(matrix(Clemson.lon.lat, 1, 2),
matrix(c(NA_lon[lon_id], NA_lat[lat_id]), 1, 2), miles = F)
== min(dist2Clemson))
```

```
##      [,1]
## [1,] TRUE
```

```
Monterey.lon.lat <- c(-121.8979, 36.5973) # (lon,lat)
dist2Monterey <- rdist.earth(
  matrix(Monterey.lon.lat, 1, 2),
  expand.grid(NA_lon, NA_lat),
  miles = F
)

id <- which.min(dist2Monterey)

(lon_id <- id %% nlon)
```

```
## [1] 5
```

```
(lat_id <- id %% nlon + 1)
```

```
## [1] 18
```

```
(rdist.earth(
  matrix(Monterey.lon.lat, 1, 2),
  matrix(c(NA_lon[lon_id], NA_lat[lat_id]), 1, 2),
  miles = F
)== min(dist2Monterey))
```

```
##      [,1]
## [1,] TRUE
```

c. Third, aggregate the daily data to the annual average time series. You could use the following script to achieve this:

```
time_series_daily <- NA_MaxT[lon_id, lat_id,]
print("Daily Time Series summary: ")
```

```
## [1] "Daily Time Series summary: "
```

```
print(summary(time_series_daily))
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      4.173  13.178  14.816  14.864  16.539  24.845
```

```
time_series_yearly <- tapply(time_series_daily, list(yr), mean)
print("Yearly Time Series summary: ")
```

```
## [1] "Yearly Time Series summary: "
```

```
print(summary(time_series_yearly))
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    14.06   14.44   14.74   14.86   15.26   16.18
```

2. Describe the main features of the annual time series data you extracted.

```
# tsfeatures for (feature autoextraction)
yearly_features <- tsfeatures(time_series_yearly)
print(yearly_features)
```

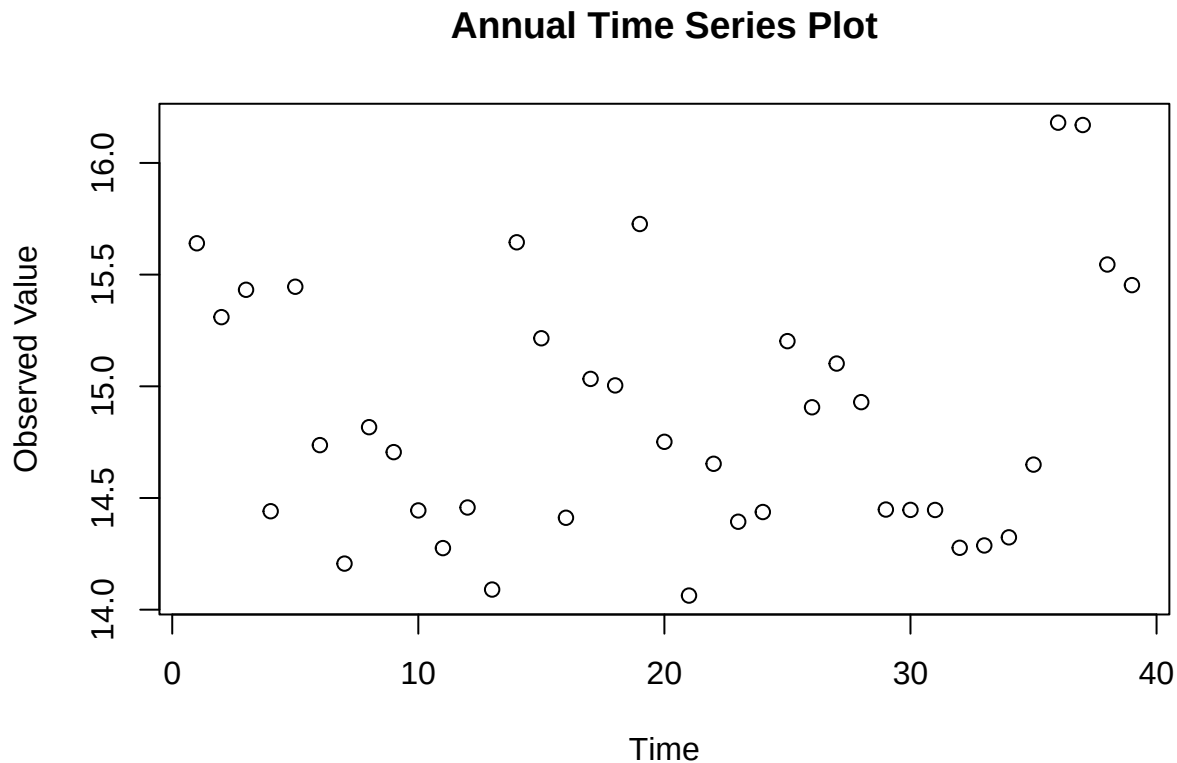
```
## # A tibble: 1 x 16
##   frequency nperiods seasonal_period trend      spike linearity curvature e_acf1
##   <dbl>     <dbl>         <dbl> <dbl>     <dbl>     <dbl>     <dbl> <dbl>
## 1         1         0             1 0.434 0.000433   0.0857     2.28 0.0822
## # i 8 more variables: e_acf10 <dbl>, entropy <dbl>, x_acf1 <dbl>,
## #   x_acf10 <dbl>, diff1_acf1 <dbl>, diff1_acf10 <dbl>, diff2_acf1 <dbl>,
## #   diff2_acf10 <dbl>
```

Our time series feature analysis using *tsfeatures* for the annual series reveals the following:

Feature	Value	Analysis
trend	0.4337	Moderately strong trend, indicates presence of long term pattern
spike	0.00042	Extremely low, indicates no significant anomalies or jumps in any given year
linearity	0.08572	Moderately low - some possibility model is linear
curvature	2.27771	High value strongly indicates non-linearity
entropy	0.9574	High value, indicates limited predictability and complex relations between the data
x_acf1	0.4117	Moderate indication of autocorrelation, any given years values show to depend on its previous year

3. Is it reasonable to assume that there is a linear trend? If so, estimate and remove the trend to get the detrended time series.

```
plot(time_series_yearly, main= "Annual Time Series Plot", ylab= "Observed Value", xlab= "Time")
```



Our graph does not reveal much significant information. However, based on our previous results from *tsfeatures*, it is still somewhat reasonable to expect the possibility of a linear trend:

1. trend: value 0.43 shows significant enough trend in data to expect some prediction.
2. linearity: while this is not 0, it is still low enough to keep linearity as a reasonable possibility
3. curvature: even though value is high (indicating a non-perfect linear fit), it is still significant

Since a linear trend is not out of question for our time series, we will now estimate its value.

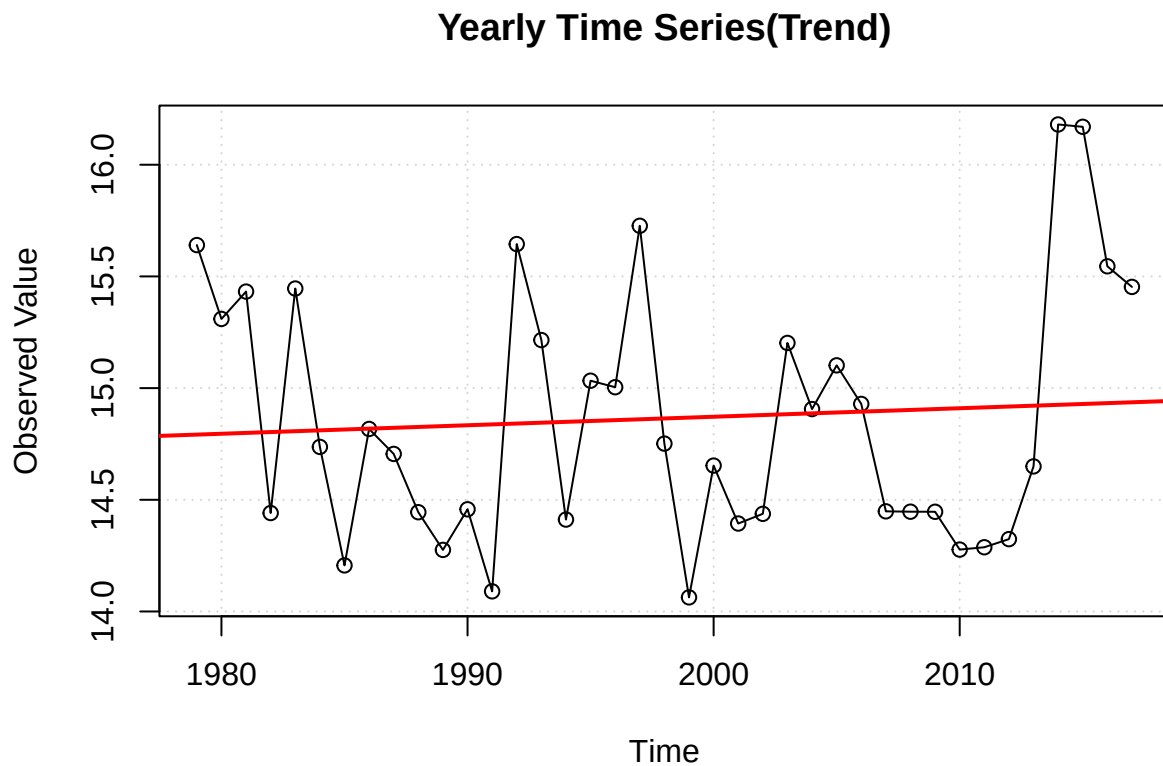
```
yrs <- unique(yr)

# fitting linear model with trend
lm_annual <- lm(time_series_yearly ~ yrs)
summary(lm_annual)
```

```
##
## Call:
```

```
## lm(formula = time_series_yearly ~ yrs)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.8047 -0.4528 -0.1126  0.4420  1.2551
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  7.254522  16.272999   0.446   0.658
## yrs          0.003809   0.008145   0.468   0.643
##
## Residual standard error: 0.5724 on 37 degrees of freedom
## Multiple R-squared:  0.005876,    Adjusted R-squared:  -0.02099
## F-statistic: 0.2187 on 1 and 37 DF,  p-value: 0.6428
```

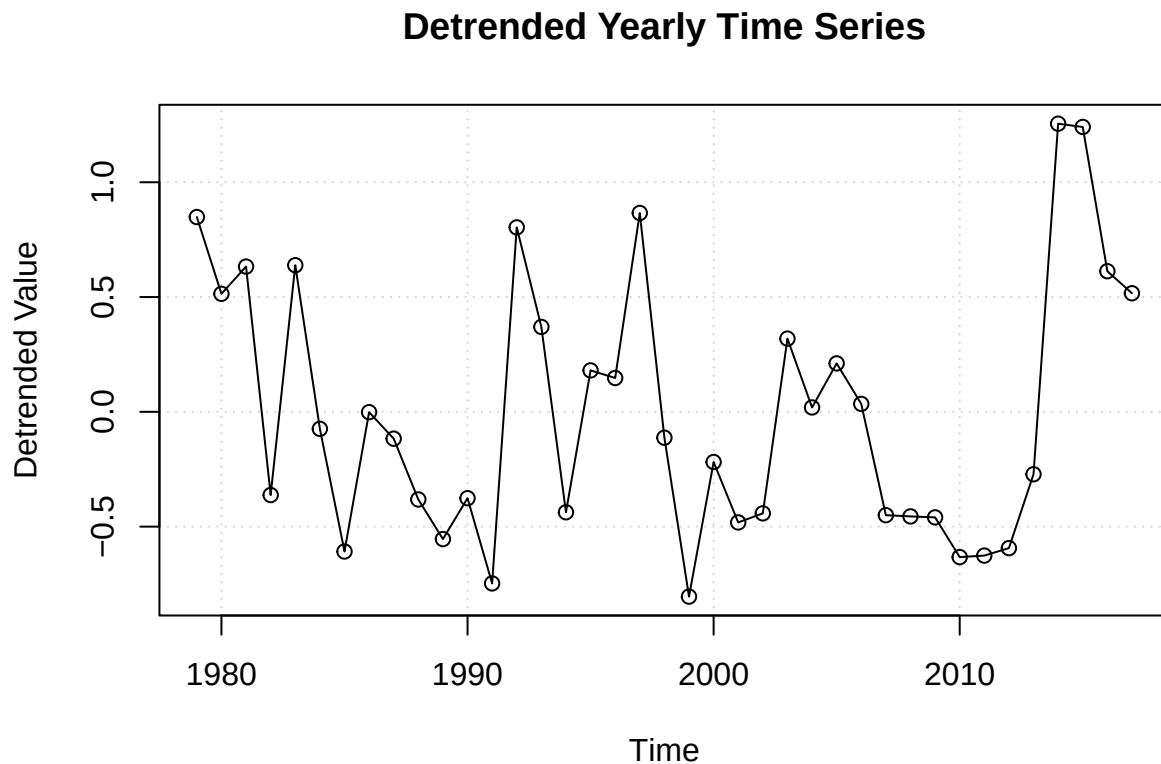
```
plot(
  x= yrs,
  y= time_series_yearly,
  type= "o", # measured points w/ time continuity
  main= "Yearly Time Series(Trend)",
  ylab= "Observed Value",
  xlab= "Time",
  panel.first= grid()
)
abline(lm_annual, col= "red", lwd= 2)
```



Our linear model fit reveals a p-value of 0.64, showing that a linear trend is statistically insignificant - leading us to fail to reject the null hypothesis and conclude there is no strong proof of linear behavior in the data. This is further confirmed by our Yearly Time Series(Trend) plot, which shows the data fluctuating significantly around our linear trend line (red) as well as many clearly non-linear patterns. It may be possible that the data could have a different measurable trend such as cyclical behavior, as is not unusual with environmental data.

Although our linear trend model was not statistically significant, we will proceed to detrend the time series data to obtain residual series which reflect deviation from potential linear trend. Our detrended model will provide useful for further decomposition or static testing. We will detrend our data by subtracting the fitted linear trend from observed values.

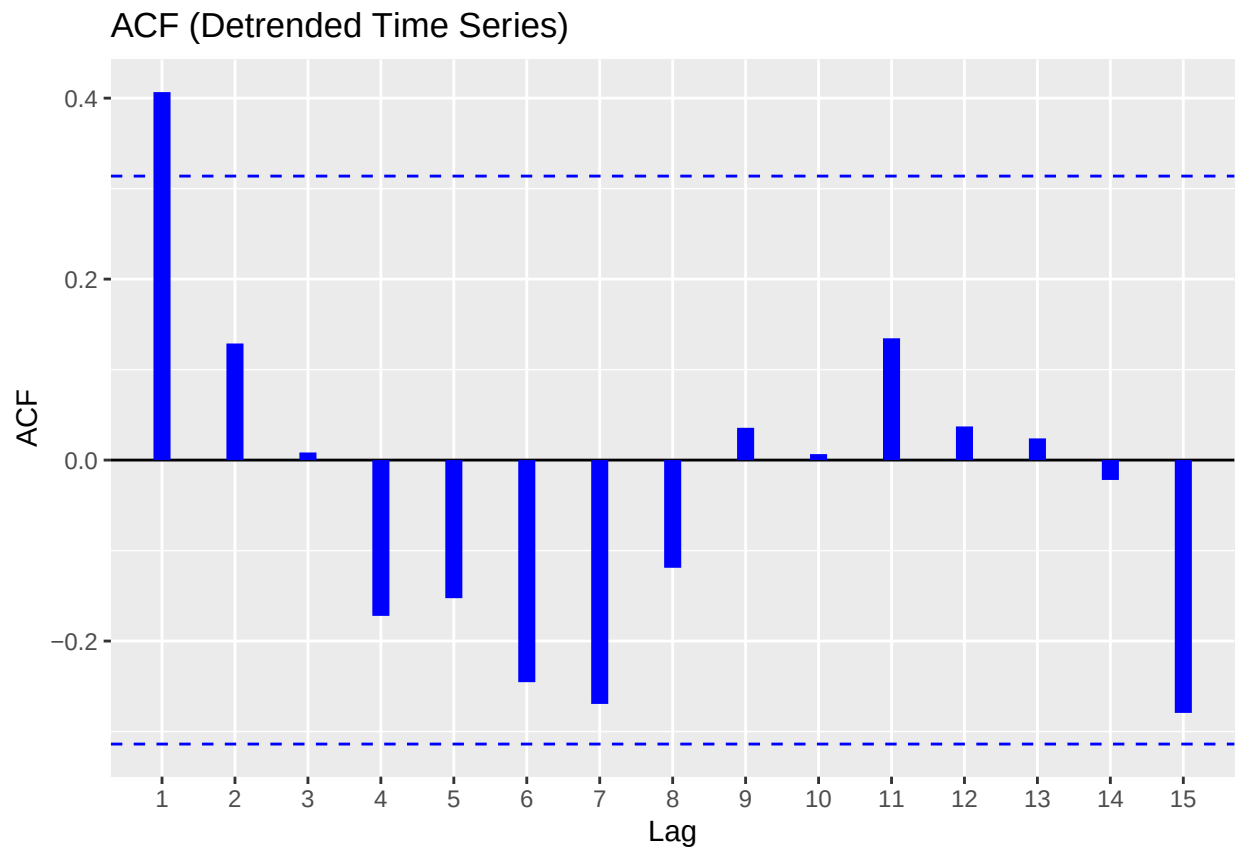
```
# detrended time series
detrended_ts <- residuals(lm_annual)
plot(
  yrs,
  detrended_ts,
  type= "o",
  main= "Detrended Yearly Time Series",
  ylab= "Detrended Value",
  xlab= "Time",
  panel.first= grid()
)
```



4. Make acf and pacf plots for the detrended time series to identify possible ARMA models. That is, determine the possible orders p for the AR component and the orders q for the MA component.

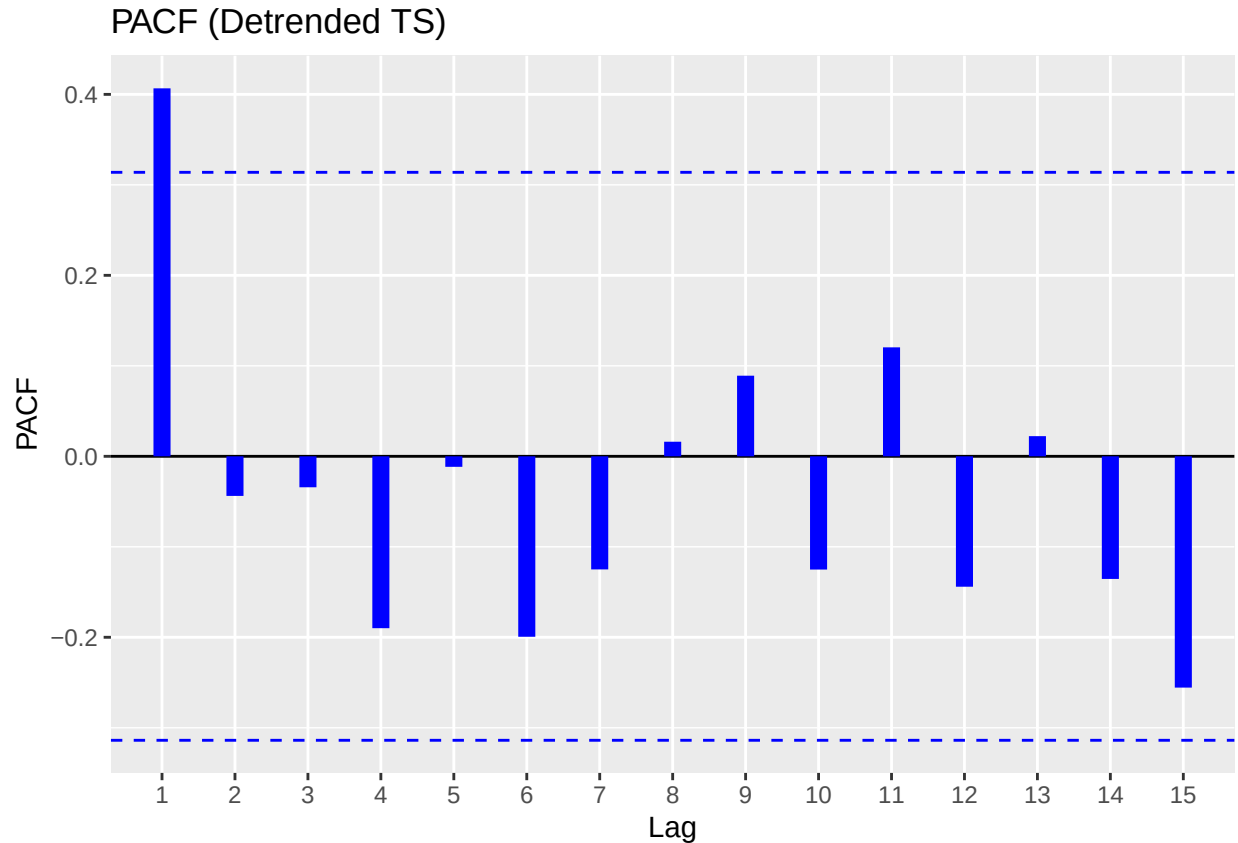
```
# ACF plot
ggAcf(detrended_ts,
      main= "ACF (Detrended Time Series)",
      size= 3, col= "blue")
```

```
## Warning in ggplot2::geom_segment(lineend = "butt", ...): Ignoring unknown
## parameters: 'main'
```



```
# PACF plot
ggPacf(detrended_ts,
      main= "PACF (Detrended TS)",
      size= 3, col= "blue")
```

```
## Warning in ggplot2::geom_segment(lineend = "butt", ...): Ignoring unknown
## parameters: 'main'
```



In determining a suitable ARMA model for our detrended time series, we examined the Auto-correlation and Partial auto-correlation function plot generated using residuals of the linear trend model.

ACF Observations:

- Shows significant spike at lag=1, which is followed by values which quickly taper off and fall within the confidence bounds.
- This cutoff after lag=1 is native to a Moving Average process, with order $q=1 \rightarrow$ suggesting a potential MA(1) component.

PACF Observations:

- Also shows significant spike at lag=1, however subsequent lags do not show substantial values.
- This quality is characteristic of Auto-regressive processes of order $p=1 \rightarrow$ suggests a potential AR(1) component.

Model Implications:

Since both ACF and PACF show prominent spike at lag=1 and no significant subsequent autocorrelation, we can establish a model as follows:

ARMA(1,1): Both MA(1) and AR(1) components are relevant.

However, we will also establish simpler models, such as $AR(1)$ and $MA(1)$ for comparing using model selection criteria (AIC/BIC).

5. Fit the possible ARMA models you identified and perform model selection to determine the ‘best’ model.

To identify our optimal ARMA model for our detrended time series, we will fit our three candidate models we found from ACF/PACF plot analysis.

- AR(1): Autoregressive model of order 1.
- MA(1): Moving average model of order 1.
- ARMA(1,1): Combined model autoregressive + moving average.

We will use the *Arima()* function from the **forecast** library to fit each ARMA model and determine optimal choice by comparing AIC/BIC values.

```
# fit AR(1), MA(1), ARMA(1,1)
model_ar1 <- Arima(detrended_ts, order= c(1,0,0))
model_ma1 <- Arima(detrended_ts, order= c(0,0,1))
model_arma11 <- Arima(detrended_ts, order= c(1,0,1))

# summary(model_arma11)

# AIC vs BIC to pick best model
model_comparison <- data.frame(
  Model = c("AR(1)", "MA(1)", "ARMA(1,1)"),
  AIC = c(AIC(model_ar1), AIC(model_ma1), AIC(model_arma11)),
  BIC = c(BIC(model_ar1), BIC(model_ma1), BIC(model_arma11))
)

print(model_comparison)
```

##	Model	AIC	BIC
## 1	AR(1)	63.54890	68.53959
## 2	MA(1)	63.97596	68.96665
## 3	ARMA(1,1)	65.51581	72.17006

Results:

- The AR(1) model has lowest AIC/BIC values by a small margin, indicating it offers the best trade off
- MA(1) very close in performance.
- ARMA(1,1) clearly not an optimal choice.

Based on model selection criteria, we will select AR(1) model as our choice for modeling this time series. This implies a linear relationship with last year’s anomaly alone best explains the current year’s temperature anomaly.

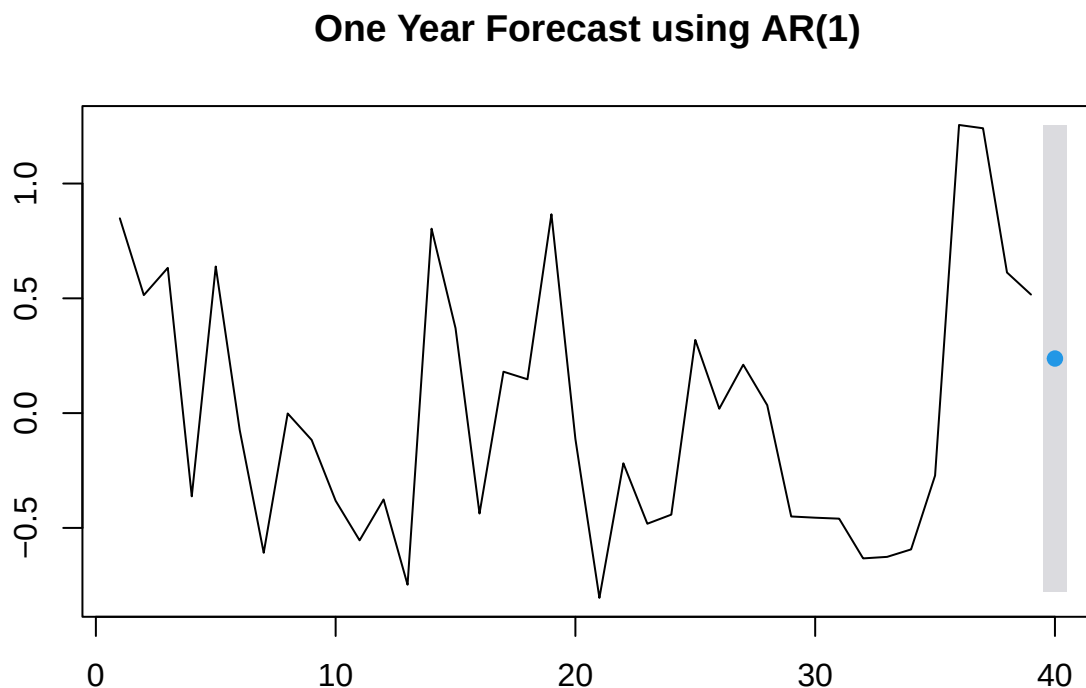
6. Perform a one-year-ahead forecast. Calculate the prediction and provide a 95% Prediction Interval.

Using the AR(1) model, we will perform a one year forecast to predict the next value in our detrended anomaly series. We will use the *forecast()* function with 'h=1' to compute forecast for one time step ahead, as well as specify a 95% Prediction Interval to quantify uncertainty around forecast.

```
forecast_ar1 <- forecast(model_ar1, h= 1, level= 95)
print(forecast_ar1)
```

```
##      Point Forecast      Lo 95      Hi 95
## 40          0.2378151 -0.7777692 1.253399
```

```
plot(forecast_ar1, main= "One Year Forecast using AR(1)")
```



Forecasting Results:

We can predict a temperature anomaly for next year to be approximately 0.2378 °C. Our 95% Prediction Interval ranges from -0.778 °C to 1.253 °C, meaning we are 95% confident values will fall in this range. The positive point forecast alludes a mild increase in temperature anomaly which continues slow upwards pattern of recent years. The wide nature of the prediction interval can be reflective of external factors that our model could not account for, and also highlights uncertainty in long term forecasting from lower order AR models, which prove to be more useful for short term predictions.

Problem 2. Analyzing Monthly Average Temperature Time Series

Monthly Average

Repeat the analysis for Problem 1 but for the monthly average time series (which can be extracted using the script below). An important component of this problem is the analysis of seasonal patterns when dealing with sub-annual data. Some techniques from Week 14 should be employed to complete this problem.

2. Describe the main features of the monthly time series data you extracted.

```
time_series_monthly <- tapply(time_series_daily, list(mon), mean)
monthly_features <- tsfeatures(time_series_monthly)

print("Monthly Time Series summary: ")
```

```
## [1] "Monthly Time Series summary: "
```

```
print(summary(time_series_monthly))
```

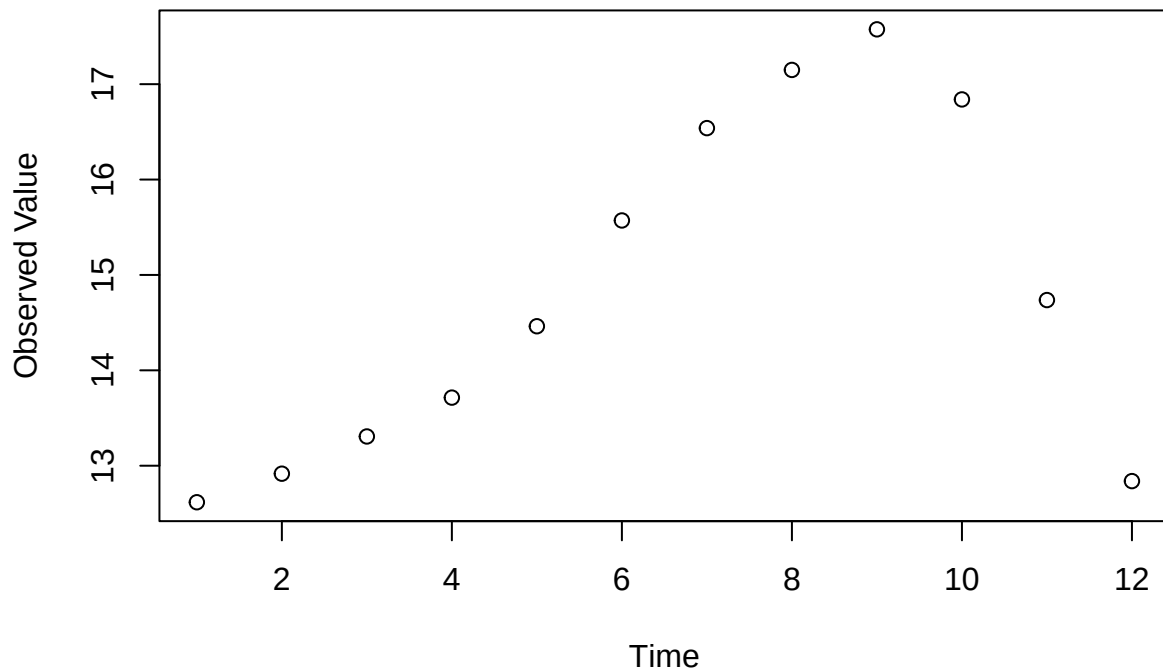
```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 12.62   13.21   14.60   14.86   16.61   17.57
```

```
print(monthly_features)
```

```
## # A tibble: 1 x 16
##   frequency nperiods seasonal_period trend      spike linearity curvature e_acf1
##   <dbl>     <dbl>         <dbl> <dbl>    <dbl>     <dbl>    <dbl> <dbl>
## 1         1         0             1 0.831 0.000345     1.33    -1.73 0.569
## # i 8 more variables: e_acf10 <dbl>, entropy <dbl>, x_acf1 <dbl>,
## #   x_acf10 <dbl>, diff1_acf1 <dbl>, diff1_acf10 <dbl>, diff2_acf1 <dbl>,
## #   diff2_acf10 <dbl>
```

```
plot(time_series_monthly, main= "Monthly Time Series Plot", ylab= "Observed Value", xlab= "Time")
```

Monthly Time Series Plot



3. Is it reasonable to assume that there is a linear trend? If so, estimate and remove the trend to get the detrended time series.

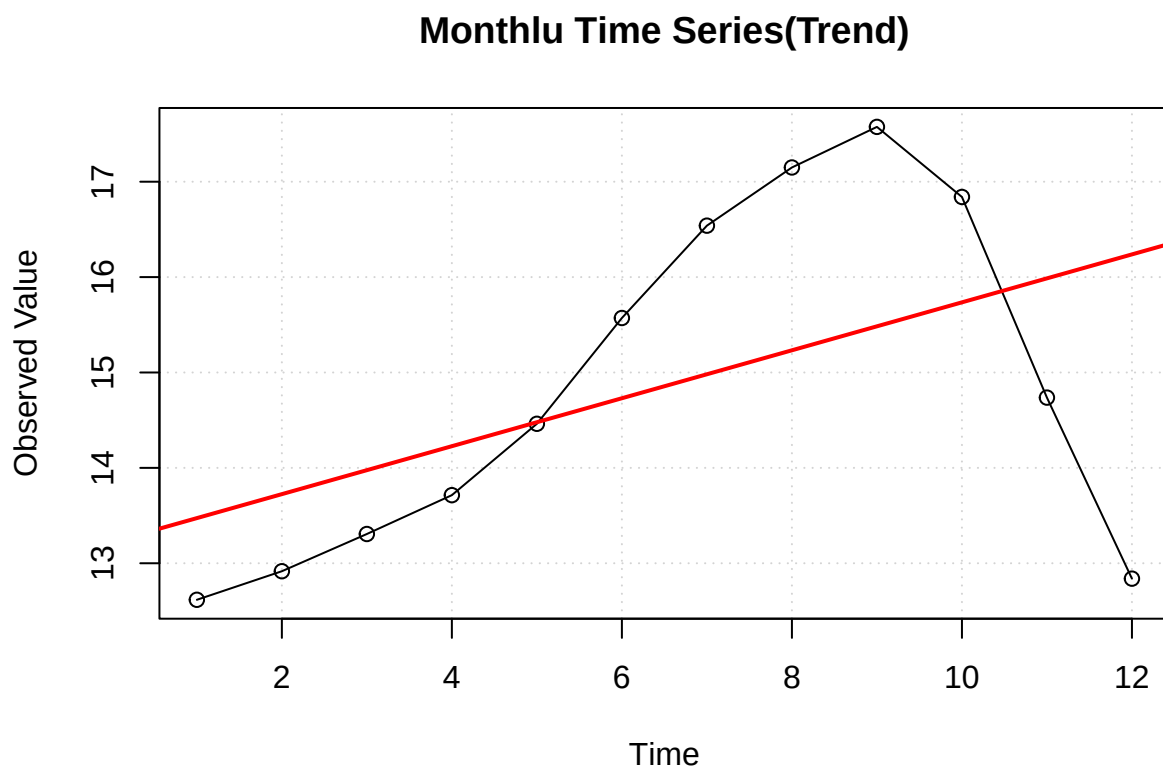
```
mons <- unique(mon)

# fitting linear model with trend
lm_monthly <- lm(time_series_monthly ~ mons)
summary(lm_monthly)

##
## Call:
## lm(formula = time_series_monthly ~ mons)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.3994 -0.8200 -0.2648  1.2182  2.0908
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  13.2218     1.0228  12.927 1.45e-07 ***
## mons          0.2514     0.1390   1.809   0.101
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.662 on 10 degrees of freedom
## Multiple R-squared:  0.2465, Adjusted R-squared:  0.1712
## F-statistic: 3.271 on 1 and 10 DF,  p-value: 0.1006
```

```
plot(
  x= mons,
  y= time_series_monthly,
  type= "o", # measured points w/ time continuity
  main= "Monthlu Time Series(Trend)",
  ylab= "Observed Value",
  xlab= "Time",
  panel.first= grid()
)
abline(lm_monthly, col= "red", lwd= 2)
```

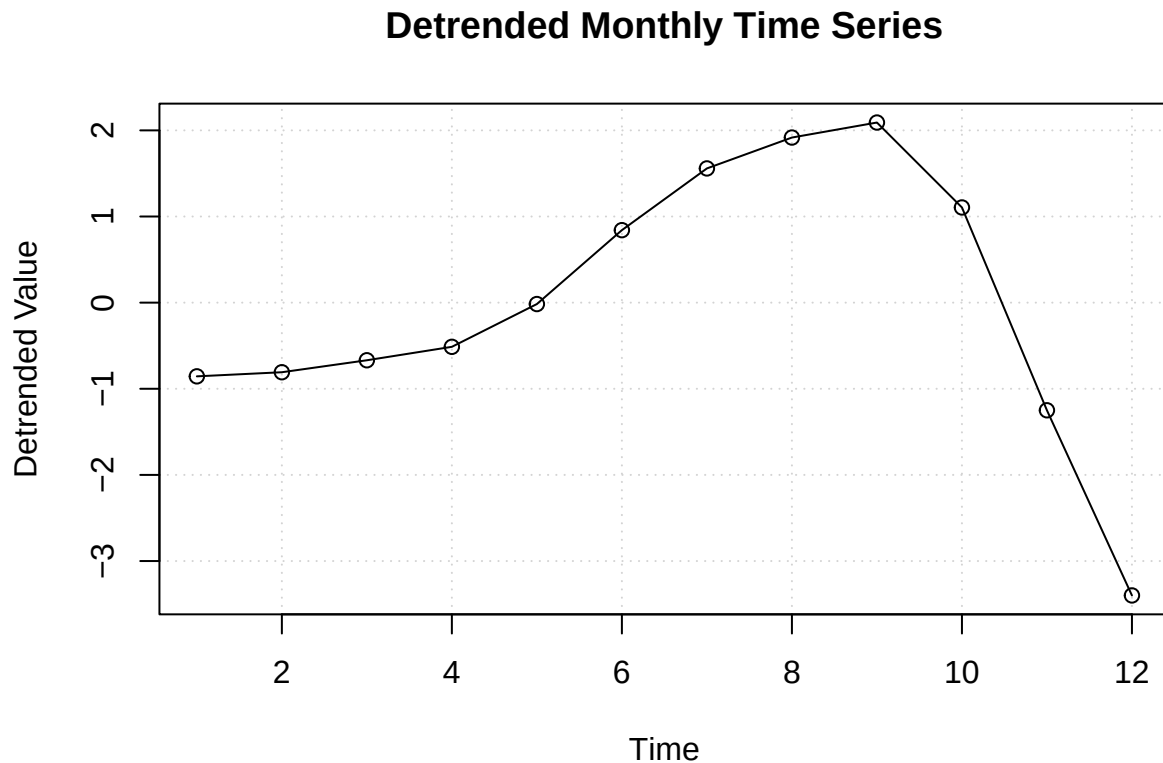


```
# detrended time series
detend_monthlyTS <- residuals(lm_monthly)
plot(
  mons,
  detend_monthlyTS,
  type= "o",
  main= "Detrended Monthly Time Series",
```

```

ylab= "Detrended Value",
xlab= "Time",
panel.first= grid()
)

```



4. Make acf and pacf plots for the detrended time series to identify possible ARMA models. That is, determine the possible orders p for the *AR* component and the orders q for the *MA* component.

```

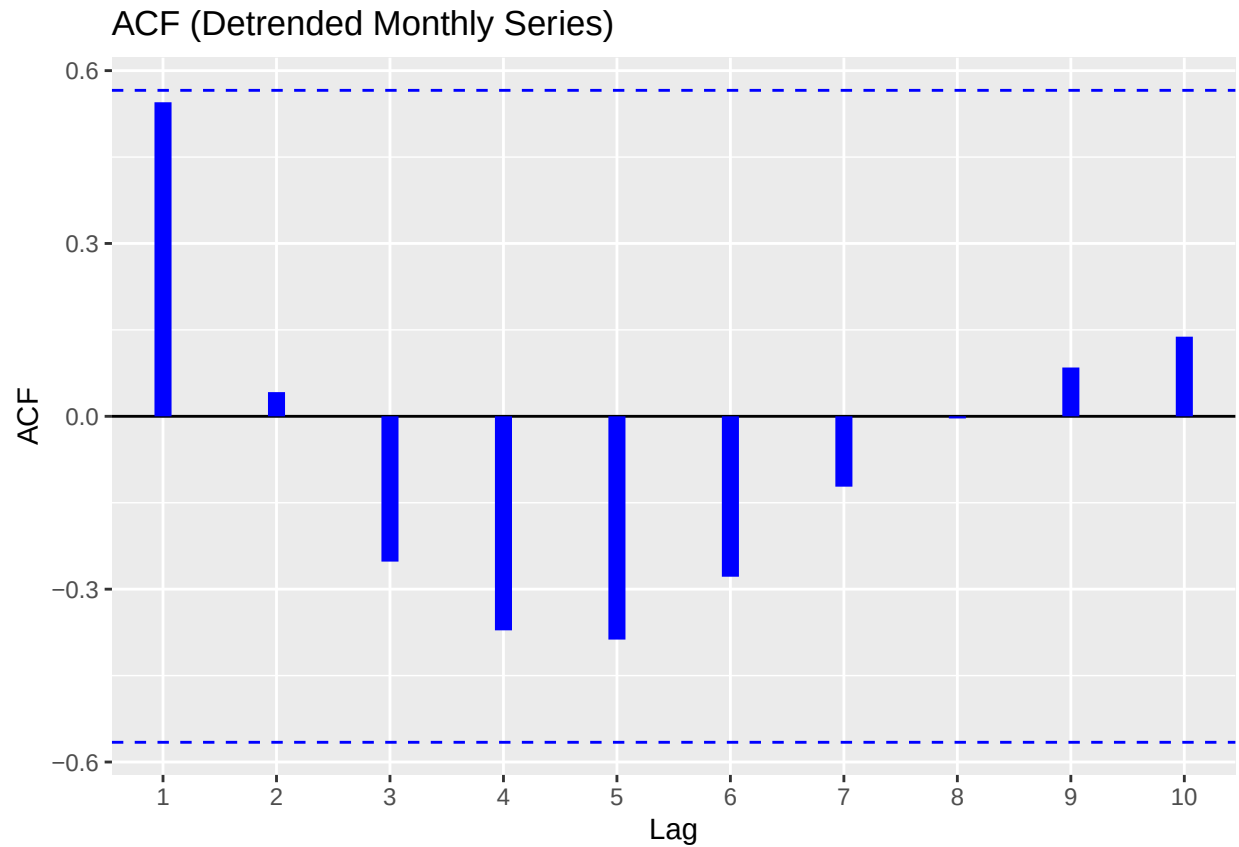
# ACF plot
ggAcf(detend_monthlyTS,
      main= "ACF (Detrended Monthly Series)",
      size= 3, col= "blue")

```

```

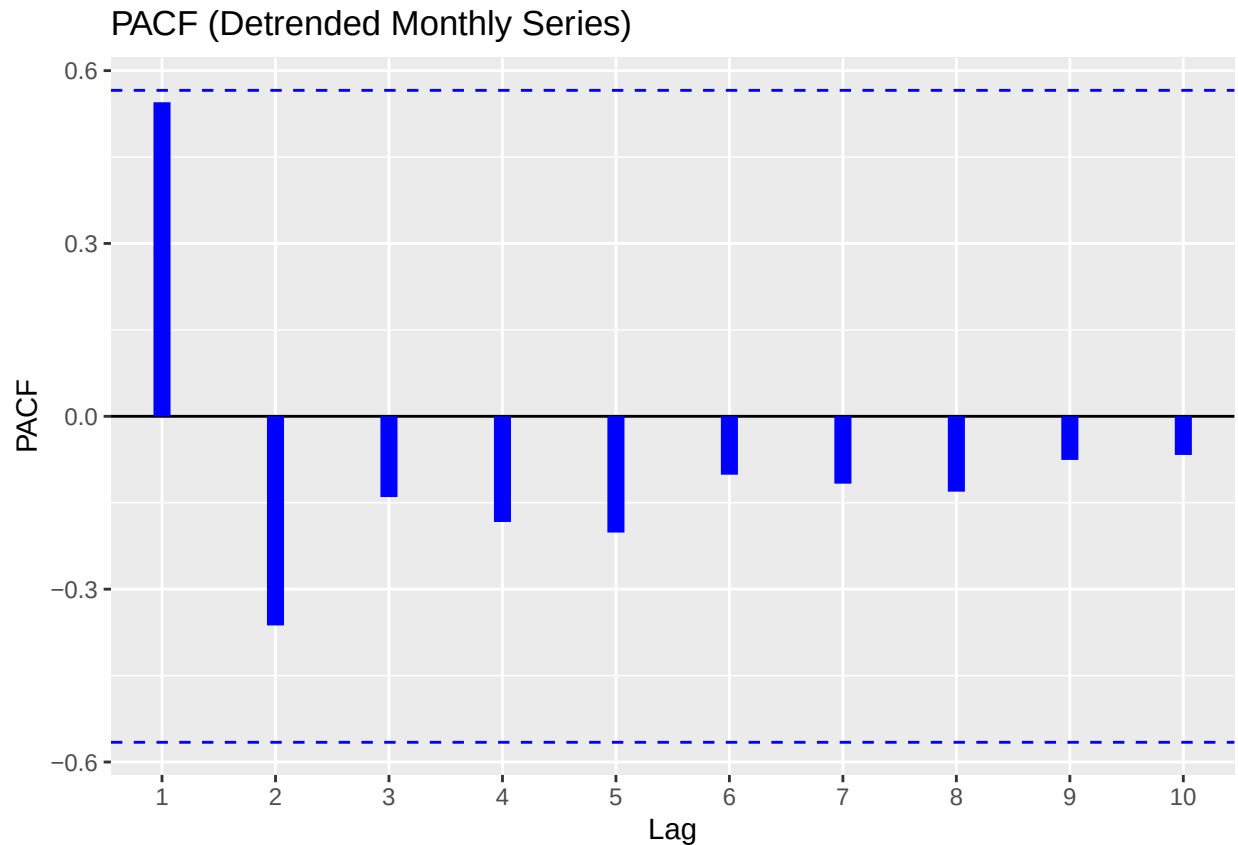
## Warning in ggplot2::geom_segment(lineend = "butt", ...): Ignoring unknown
## parameters: 'main'

```



```
# PACF plot
ggPacf(detend_monthlyTS,
  main= "PACF (Detrended Monthly Series)",
  size= 3, col= "blue")
```

```
## Warning in ggplot2::geom_segment(lineend = "butt", ...): Ignoring unknown
## parameters: 'main'
```



5. Fit the possible ARMA models you identified and perform model selection to determine the ‘best’ model.

```
# fit AR(1), MA(1), ARMA(1,1)
model_ar1 <- Arima(detend_monthlyTS, order= c(1,0,0))
model_ma1 <- Arima(detend_monthlyTS, order= c(0,0,1))
model_arma11 <- Arima(detend_monthlyTS, order= c(1,0,1))

# summary(model_arma11)

# AIC vs BIC to pick best model
model_comparison <- data.frame(
  Model = c("AR(1)", "MA(1)", "ARMA(1,1)"),
  AIC = c(AIC(model_ar1), AIC(model_ma1), AIC(model_arma11)),
  BIC = c(BIC(model_ar1), BIC(model_ma1), BIC(model_arma11))
)

print(model_comparison)
```

```
##      Model      AIC      BIC
## 1    AR(1) 41.81017 43.26489
```

```
## 2      MA(1) 40.37756 41.83228
## 3 ARMA(1,1) 33.74817 35.68779
```